Credit Risk Classification (Loan Default Prediction)

By:

Matheus Santos Gomes

Joaquin Hidalgo-Estrada

Rajan Jinadra

Natiana Urdaneta Barazarte

Introduction:

Our project focuses on **Credit Risk Classification**, a machine learning project designed to predict whether a borrower is likely to default on a loan. In this project, default is labeled as **1**, meaning the borrower is considered risky and may not repay the loan as agreed. Not default is labeled as **0**, meaning the borrower is considered a safer customer who is more likely to repay the loan. The goal of this project is to use historical financial and demographic data to identify borrower behavior patterns and support better lending decisions.

The dataset used for this project contains **30,000 clients** and **23 features**. These features include important borrower information such as credit limit, payment history, bill amounts, and demographic variables. Since the target variable separates borrowers into default and non-default groups, this dataset is well-suited for building classification models that predict credit risk.

This project is motivated by the real-world importance of loan decision-making. Poor loan decisions can create serious financial problems for both banks and borrowers. For lenders, approving too many risky borrowers can lead to unpaid loans and financial losses. For borrowers, receiving a loan they cannot afford may lead to debt and long-term financial stress. Because of this, credit risk models can help make loan approvals more consistent, faster, and data-driven.

A key idea in this project is that identifying a **good customer** can be just as important as identifying a risky customer. In the real world, banks not only want to avoid losses from defaulters; they want to approve borrowers who will repay their loans while generating profit through interest. For example, if a customer uses a million-dollar line of credit, the bank benefits

greatly from the interest earned. Therefore, correctly identifying **class 0, non-default customers**, is very valuable because rejecting borrowers could mean losing profitable and consistent customers.

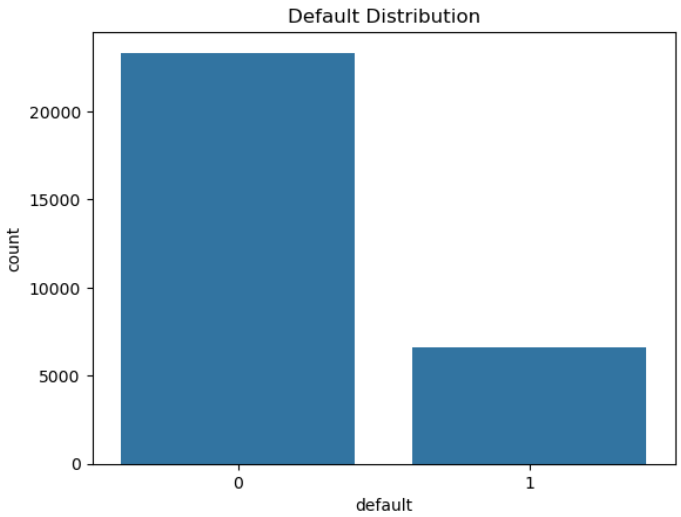
To study this problem, we compared four different classification models: **XGBoost, Naive Bayes, Logistic Regression, and a Feed Forward Neural Network**. Each model was used to predict borrower default risk and compare performance using metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. These models allowed us to examine the tradeoff between detecting risky borrowers and correctly approving safe borrowers.

The key problem addressed in this project is: **How can financial and demographic borrower data be used to predict whether a customer will default or not default on a loan?** Through data cleaning, exploratory data analysis, feature engineering, model building, and performance evaluation, this project aims to show how machine learning can support smarter lending decisions. Overall, the project connects statistical classification methods with a practical financial problem and highlights how credit risk prediction can help banks reduce losses while also identifying valuable customers.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the structure of the dataset, identify patterns in borrower behavior, and guide feature engineering and model selection. As mentioned earlier in the introduction, the dataset contains 30,000 observations with financial, demographic, and payment history variables. We downloaded the dataset from Kaggle using the UCI Credit Card Default dataset (Default of Credit Card Clients Dataset) and loaded it into Python using pandas.

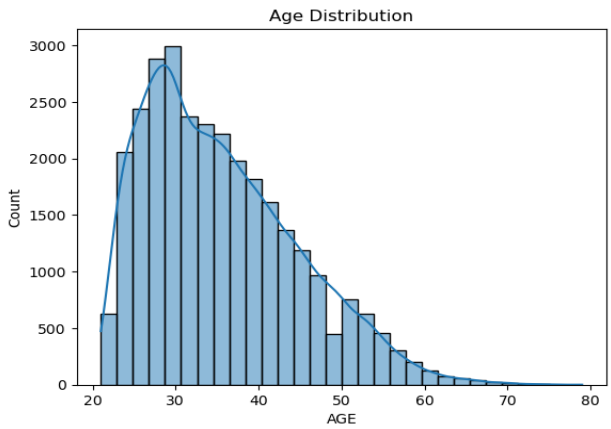
The distribution of the target variable (default vs. non-default):



The distribution of the target variable reveals a clear class imbalance, with most borrowers (78%) classified as non-default (0) and approximately 22% classified as default (1). This pattern reflects real-world lending environments, where most borrowers repay their loans. However, this imbalance presents a challenge for model development, as predictive models may become biased toward the majority class. For this reason, evaluation metrics such as precision, recall, F1-score, and ROC-AUC are more appropriate than accuracy alone. This observation also

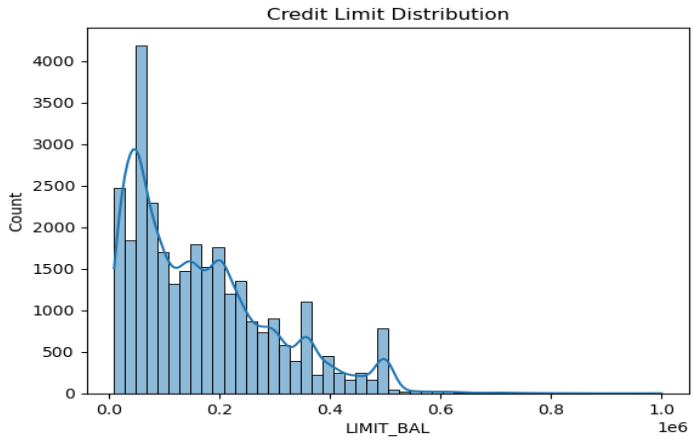
motivates the use of techniques such as class weighting and threshold tuning in later stages of the analysis.

The age distribution of borrowers:



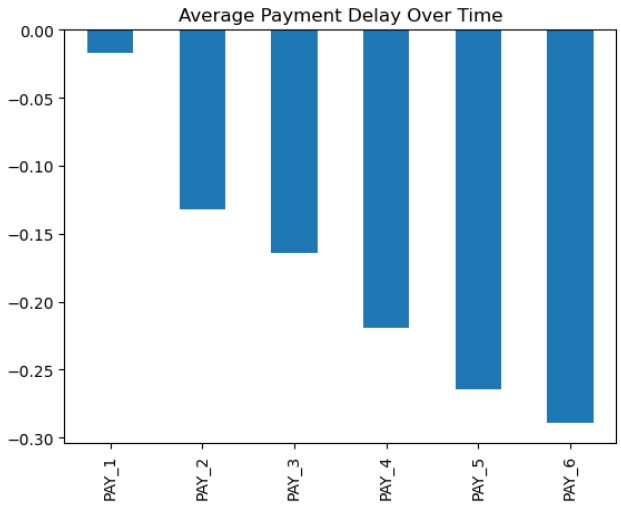
The age distribution of borrowers shows a right-skewed pattern, with most individuals concentrated between approximately 25 and 40 years old, and fewer observations at higher ages. This indicates that the dataset is dominated by younger to middle-aged borrowers. While age provides useful demographic context, it is not expected to be a strong standalone predictor of default risk when compared to behavioral variables such as payment history. Nonetheless, it may still contribute to model performance when combined with other features.

The distribution of credit limits (LIMIT_BAL):



The distribution of credit limits (LIMIT_BAL) is heavily right-skewed, with most borrowers having relatively low to moderate credit limits and a small number of individuals with very high limits. This indicates the presence of outliers and significant variability in borrowing capacity. Such skewness can affect certain machine learning models, particularly those sensitive to scale and extreme values. As a result, preprocessing steps such as scaling are important, and models that are robust to skewed distributions, such as tree-based methods, are particularly well-suited for this dataset.

The average payment delay across six months (PAY_1 to PAY_6):



An analysis of payment behavior over time reveals important patterns in borrower activity. The average payment delay across the six observed months (PAY_1 through PAY_6) is generally negative, indicating that many borrowers tend to pay on time or early. However, there is a gradual increase in delay in later months, suggesting that some borrowers experience worsening payment behavior over time. This temporal pattern highlights the importance of incorporating payment history into the predictive modeling process. It also justifies the creation of engineered features such as MAX_DELAY and LATE_COUNT, which summarize payment

behavior across multiple periods and provide stronger predictive signals than individual monthly variables.

Overall, the EDA process reveals several important characteristics of the dataset. The target variable is imbalanced, with a higher proportion of non-default cases, which reflects real-world lending patterns but introduces challenges for classification. Financial variables, such as credit limits and bill amounts, are highly skewed and contain noticeable outliers, indicating substantial variability in borrower financial behavior. Additionally, strong relationships exist among financial features across time, suggesting the presence of correlation rather than independence. Most importantly, payment history variables show clear patterns associated with default risk, with increasing delays and inconsistent payment behavior indicating a higher likelihood of default. These findings highlight the importance of behavioral features and provide a strong foundation for feature engineering and subsequent modeling.

Data Cleaning

Before building the credit default prediction models, we cleaned and prepared the dataset to make sure it was ready for analysis. The original dataset contained **30,000 rows and 25 columns**, including borrower demographic information, credit limit, monthly bill amounts, payment amounts, repayment history, and the target variable.

The first step was to inspect the dataset using functions such as **df.shape, df.head(), df.info(),** and **df.describe()**. This helped us understand the structure of the data, the number of observations, the data types of each column, and the general range of the numerical values. After

reviewing the dataset, we checked for missing values using **df.isnull().sum()**. The dataset did not contain any missing values, so no rows or columns had to be removed for missing data.

Next, we removed the ID column because it only served as an index number for each customer and did not provide any useful predictive information for the model. Keeping this column could have added unnecessary noise to the model, so it was dropped from the dataset.

We also renamed some columns to make the dataset easier to understand. The target variable, originally named default.payment.next.month, was renamed to default. This made the target column simpler to reference throughout the project. The column **PAY_0** was also renamed to **PAY_1** so that the repayment status columns followed a clearer monthly order from **PAY_1** to **PAY_6.**

After that, duplicate rows were checked and removed using **df.drop_duplicates()**. This step was important because duplicate records can overrepresent certain borrowers and potentially bias the model results. Removing duplicates helped make sure each observation was treated as a unique record.

Some categorical values also needed to be cleaned. In the EDUCATION column, the values 0, 5, and 6 represented unclear or unknown education categories. These were combined into one category labeled 5, while keeping the separate “other” category unchanged. Similarly, in the MARRIAGE column, the value 0 was not a valid standard category, so it was replaced with 3, which represents the “other” category. These changes helped make the categorical variables more consistent and easier to interpret.

We then examined the distribution of the target variable using a count plot and value counts. The results showed that the dataset was imbalanced, with about **77.88% of customers classified as non-default** and **22.12% classified as default**. This is important because a model

could achieve high accuracy by mostly predicting the majority class. Because of this imbalance, later model evaluation should focus on more than accuracy alone, including precision, recall, F1-score, ROC-AUC, and confusion matrices.

We also examined the distributions of important numerical variables such as AGE and LIMIT_BAL. Both variables showed some right skewness, especially credit limit. To help reduce skewness and make the data more suitable for modeling, log transformations were created for these variables using np.log1p(). This produced new columns called AGE_LOG and LIMIT_BAL_LOG.

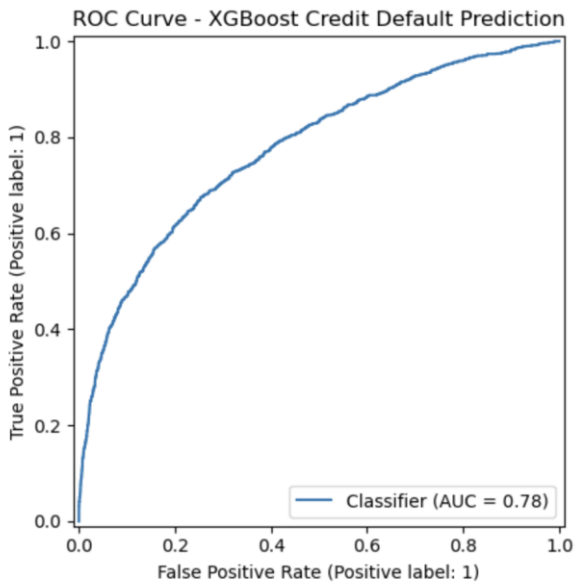
Finally, several new features were created to capture borrower payment behavior better. The six-monthly bill amount columns were summed to create TOTAL_BILL, and the six-monthly payment amount columns were summed to create TOTAL_PAY. A PAY_RATIO variable was also created by dividing total payments by total bills, with +1 added to the denominator to avoid division by zero. In addition, a UTILIZATION variable was created by dividing the total bill amount by credit limit. These engineered features help represent how much debt a borrower carries compared to their available credit and how much they are paying back relative to what they owe.

After completing these cleaning and feature engineering steps, the final cleaned dataset was saved as a CSV file called cleaned_credit_data_v1.csv. This cleaned dataset was then ready for exploratory data analysis and model training.

XGBoost Model:

Model Results

The XGBoost model achieved the strongest overall performance among all models evaluated in this study. At the default classification threshold of 0.50, the model obtained an accuracy of approximately 76%, with a precision of 0.466, a recall of 0.615, and an F1-score of 0.529. The ROC-AUC score of 0.776 indicates strong discriminative ability, meaning the model can effectively distinguish between defaulters and non-defaulters across different thresholds.



Compared to simpler models such as Logistic Regression and Naïve Bayes, XGBoost performs better because it captures complex, non-linear relationships in the data, particularly patterns related to borrower payment behavior.

Threshold Adjustment

To further refine model performance, classification threshold tuning was applied. The optimal threshold was identified as 0.55.

At this optimized threshold:

- Precision increased from 0.466 to 0.508
- Recall decreased slightly from 0.615 to 0.563
- F1-score improved from 0.529 to 0.534
- ROC-AUC remained constant at 0.776

This adjustment reflects a shift toward a more conservative prediction strategy. The model becomes more selective when labeling borrowers as defaulters, reducing false positives while slightly increasing false negatives. This tradeoff is important in financial applications, where incorrectly rejecting reliable borrowers can result in lost profit.

```
XGBoost Results - Default Threshold 0.50
========================================
Precision : 0.4657
Recall    : 0.6146
F1-Score  : 0.5299
ROC-AUC   : 0.7756

Confusion Matrix:
[[3732  935]
 [ 511  815]]

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.80      0.84      4667
           1       0.47      0.61      0.53      1326

    accuracy                           0.76      5993
   macro avg       0.67      0.71      0.68      5993
weighted avg       0.79      0.76      0.77      5993
```

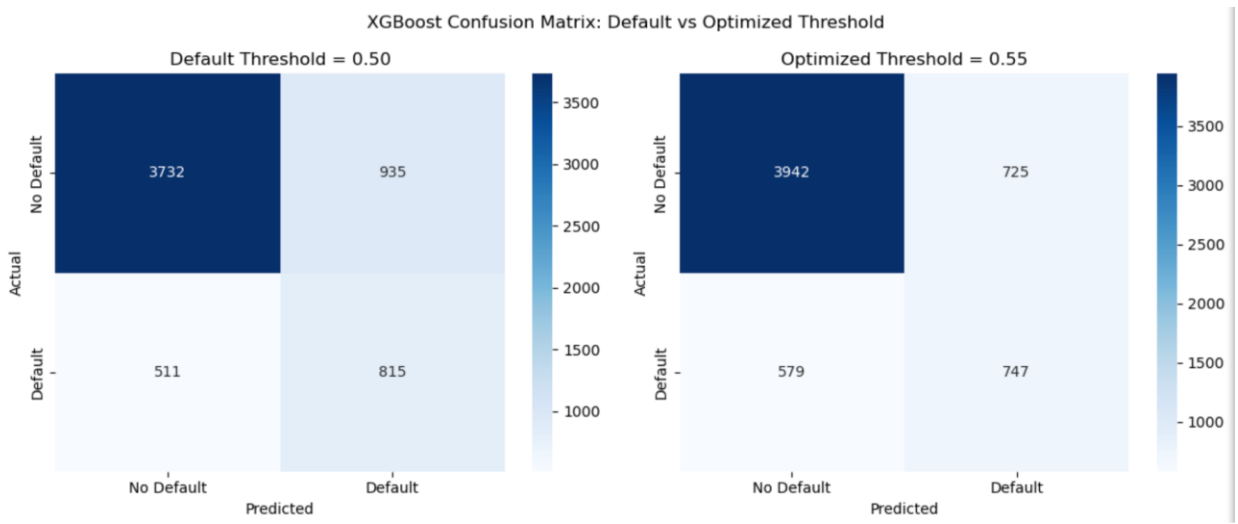
```
XGBoost Results - Optimized Threshold
=====================================
Best Threshold: 0.55
Precision     : 0.5075
Recall        : 0.5633
F1-Score      : 0.534
ROC-AUC       : 0.7756

Optimized Confusion Matrix:
[[3942  725]
 [ 579  747]]

Optimized Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.84      0.86      4667
           1       0.51      0.56      0.53      1326

    accuracy                           0.78      5993
   macro avg       0.69      0.70      0.70      5993
weighted avg       0.79      0.78      0.79      5993
```

Confusion Matrix Analysis

The confusion matrix provides further insight into this tradeoff. After threshold optimization:

- True negatives increased (3732 → 3942), improving the identification of safe borrowers
- False positives decreased (935 → 725), reducing incorrect risk classification
- False negatives increased slightly (511 → 579), indicating a small increase in missed defaulters

These results show that threshold tuning improves the model's reliability when predicting defaults, while slightly reducing its sensitivity to all risky cases.

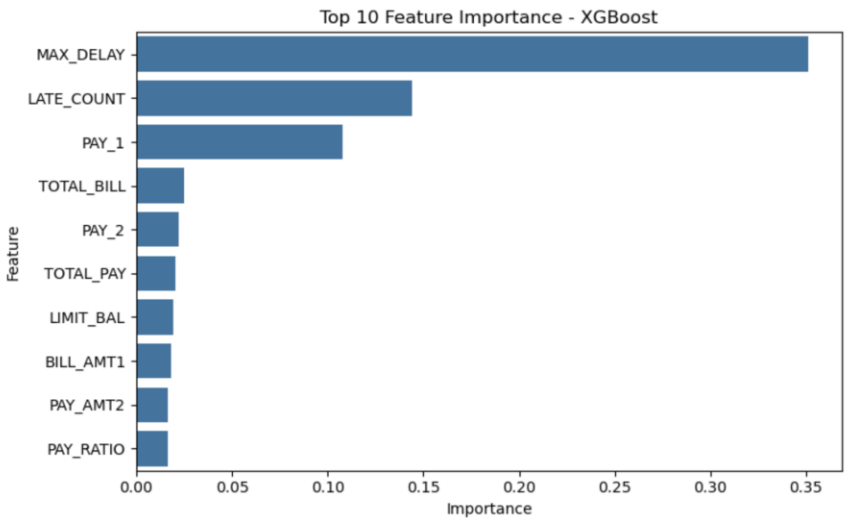


Feature Importance

Feature importance analysis highlights that the most influential predictors are:

- **MAX_DELAY (0.351)**
- **LATE_COUNT (0.144)**
- **PAY_1 (0.108)**

These variables are directly related to payment behavior, confirming that past delays are the strongest indicators of future default risk.



It is important to note that MAX_DELAY and LATE_COUNT are engineered features derived from PAY_1 through PAY_6. Their high importance demonstrates that aggregating payment patterns provides more predictive power than using individual monthly variables alone. This emphasizes the value of feature engineering in improving model performance.

Strengths and Limitations

One of the key strengths of XGBoost is its ability to model complex relationships and interactions within the data, resulting in improved predictive accuracy compared to baseline models. Additionally, threshold optimization allowed the model to better balance precision and recall, making it more suitable for real-world decision-making.

However, the model is not without limitations. The slight reduction in recall after threshold optimization indicates that some defaulters may still be missed. This highlights the

inherent tradeoff in classification problems, particularly in imbalanced datasets such as credit risk.

Interpretation

Overall, XGBoost provides the most effective balance between identifying defaulters and avoiding incorrect classification of safe borrowers. Its strong performance, combined with its ability to leverage engineered features, makes it the most suitable model for this credit risk classification task.

Naive Bayes:

For one of our classification models, we used a **Naive Bayes classifier** to predict whether a borrower would default on a loan. Naive Bayes is a probability-based machine learning model that uses Bayes' Theorem to classify observations into different groups.

We included Naive Bayes because it is simple, fast, and useful as a baseline model. Since credit risk prediction is based on estimating the chance that a borrower may default, Naive Bayes fits the problem in a basic probabilistic way. However, the model assumes that the input features are independent of each other. This assumption is a limitation for our dataset because financial variables such as bill amounts, payment history, credit limit, and repayment behavior are often related to each other.

Model Results

After training and testing the Naive Bayes model, the results showed that the model was not very balanced. The model achieved an **accuracy of about 35.44%**, meaning it correctly

classified only about one-third of the test observations. Its **precision was about 24.27%**, which means that when the model predicted a borrower would default, it was correct only about one-fourth of the time.

The model had a very high **recall of about 90.42%**, meaning it successfully caught most of the actual defaulters. However, this high recall came with a major problem: the model incorrectly labeled many safe borrowers as defaulters. The **F1-score was about 0.3826**, showing that the model struggled to balance precision and recall. The model also achieved an **ROC-AUC of about 0.7339**, which suggests that it was learning some separation between defaulters and non-defaulters, even though the final classification results were poor.

```
Naive Bayes Results
========================================
Accuracy : 0.3544
Precision: 0.2427
Recall   : 0.9042
F1 Score : 0.3826
ROC-AUC  : 0.7339

Confusion Matrix:
[[ 925 3742]
 [ 127 1199]]

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.20      0.32      4667
           1       0.24      0.90      0.38      1326

    accuracy                           0.35      5993
   macro avg       0.56      0.55      0.35      5993
weighted avg       0.74      0.35      0.34      5993
```

This confusion matrix shows the main weakness of the model. It correctly identified **1199 actual defaulters**, which explains the high recall. However, it also incorrectly classified **3742 non-default borrowers as defaulters**. This is a serious issue because these borrowers were actually safe customers, but the model marked them as risky.

In a real banking situation, this would be a major problem. If the bank used this model, it would reject or flag a large number of good customers. Since identifying reliable non-default

customers is important for bank profit, especially customers who borrow large amounts and repay with interest, this model would not be practical as a final credit risk model.

Threshold Adjustment

We also tested an adjusted classification threshold of **0.35** to see if the model's performance improved. However, the results showed no real improvement. At this threshold, the model had a **precision of about 23.93%**, a **recall of about 91.33%**, and an **F1-score of about 0.3793**.

After threshold adjustment, the model caught slightly more defaulters, increasing true positives from **1199 to 1211**. However, it also increased false positives from **3742 to 3849**, meaning even more safe borrowers were incorrectly labeled as risky. Because of this, lowering the threshold did not solve the main issue.

```
Adjusted Threshold Results (threshold = 0.35)
========================================
Precision: 0.2393
Recall   : 0.9133
F1 Score : 0.3793

Adjusted Confusion Matrix:
[[ 818 3849]
 [ 115 1211]]

Cross-Validated ROC-AUC
========================================
Fold Scores: [0.75110294 0.73038796 0.73678916 0.7586016  0.74405222]
Mean ROC-AUC: 0.7442
Std Dev     : 0.0100
```

Cross-Validation Results

To check whether the model was learning consistently, we also used cross-validation with ROC-AUC. The fold scores ranged from about **0.73 to 0.76**, with a **mean ROC-AUC of 0.7442** and a **standard deviation of 0.0100**.

These results show that the model's ranking ability was fairly consistent across different folds. The small standard deviation means the model was stable during cross-validation. However, the main problem was not that the model failed to learn completely. Instead, the issue was that its classification behavior was poor because it was heavily biased toward predicting class **1**, or default.

Interpretation of Naive Bayes Results

Overall, the Naive Bayes model provides a useful starting point for understanding credit risk classification. Its main strength is that it is easy to interpret and computationally efficient. It can quickly classify borrowers and give probability-based predictions, making it a helpful baseline model.

However, it was not strong enough to be selected as the final model. Its biggest strength was its ability to catch actual defaulters, shown by the high recall of about **90%**. However, this came at the cost of extremely low precision and a very high number of false positives.

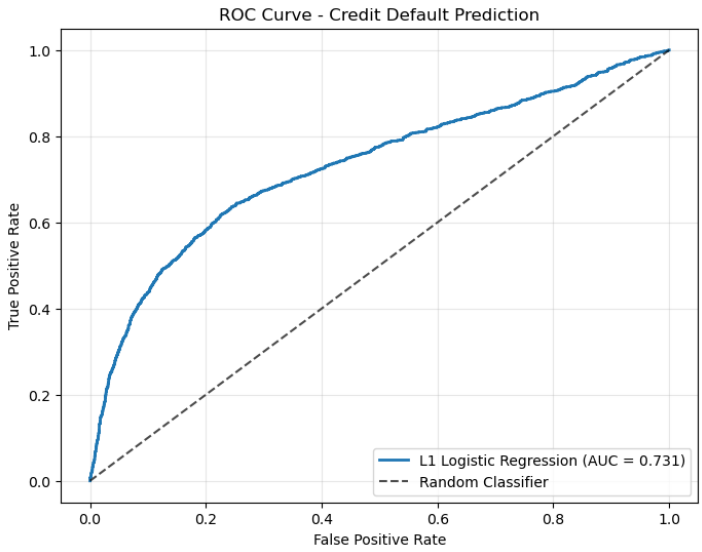
For this project, that tradeoff is not ideal because classifying good borrowers correctly is also very important. A model that wrongly flags too many non-default customers could cause the bank to lose profitable borrowers. Therefore, while Naive Bayes helped us compare a simple probability-based model against Logistic Regression, XGBoost, and the Feed Forward Neural Network, it was not the best model for this credit risk classification problem.

Logistic Regression

Logistic Regression was implemented as a core statistical model to predict borrower default risk. This model is widely used in credit risk analysis due to its interpretability and ability to estimate probabilities of default. By applying a logistic (sigmoid) function to a linear combination of input features, the model produces outputs between 0 and 1, which can be interpreted as the likelihood of default.

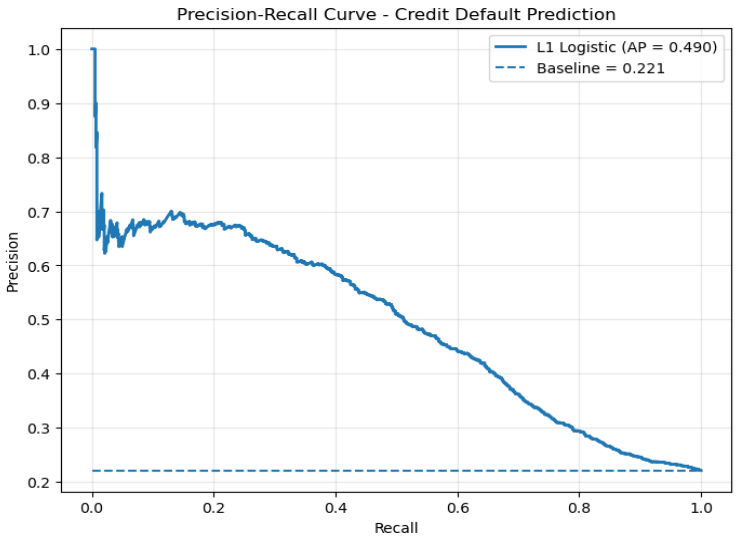
To further optimize model performance, threshold tuning was applied to adjust the classification cutoff and improve prediction quality. Rather than relying solely on the default threshold of 0.50, multiple thresholds were evaluated to better balance precision and recall. At the default threshold, the model achieved a precision of 44% and a recall of 62% for identifying defaulters. Increasing the threshold to 0.60 improved precision to 53% while reducing recall to 49%, with overall accuracy increasing from 74% to 79%. This resulted in a more conservative classification strategy that reduces false positives and minimizes the risk of incorrectly flagging reliable borrowers as high risk, while still identifying a substantial number of true defaulters. This tradeoff provides a more practical balance for lending decisions.

ROC Curve - Credit Default Prediction:



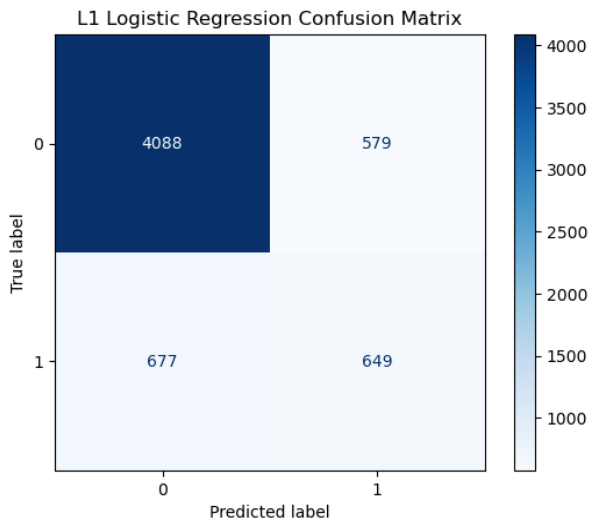
The performance of the Logistic Regression model was first evaluated using the Receiver Operating Characteristic (ROC) curve. The model achieved an ROC-AUC score of approximately 0.731, indicating a good ability to distinguish between defaulters and non-defaulters. The ROC curve consistently lies above the diagonal baseline, demonstrating that the model performs significantly better than a random classifier across all classification thresholds.

Precision-Recall Curve - Credit Default Prediction:



Further evaluation was conducted using the Precision-Recall (PR) curve. This analysis is particularly important given the class imbalance present in the dataset. The model achieved an average precision (AP) score of approximately 0.490, which is substantially higher than the baseline of 0.221. This indicates that the model is effective at identifying default cases while maintaining reasonable precision. As recall increases, precision gradually decreases, highlighting the tradeoff between capturing more defaulters and introducing false positives.

Logistic Regression Confusion Matrix:



The confusion matrix provides additional insight into the model's classification performance at the default threshold of 0.50. At this threshold, the model correctly classified 4088 non-default borrowers (true negatives) and 649 default borrowers (true positives), while also producing 579 false positives and 677 false negatives. These results reflect a moderate balance between identifying defaulters and avoiding incorrect classification of safe borrowers.

Overall, the Logistic Regression model demonstrates strong and consistent performance as a baseline classifier. It provides a solid balance between precision and recall while

maintaining good discriminatory power, as reflected in both the ROC and Precision-Recall curves. Additionally, its interpretability makes it particularly valuable in a financial context, where understanding the factors driving predictions is essential. While more complex models may capture nonlinear relationships more effectively, Logistic Regression remains a reliable and informative approach for credit risk prediction.

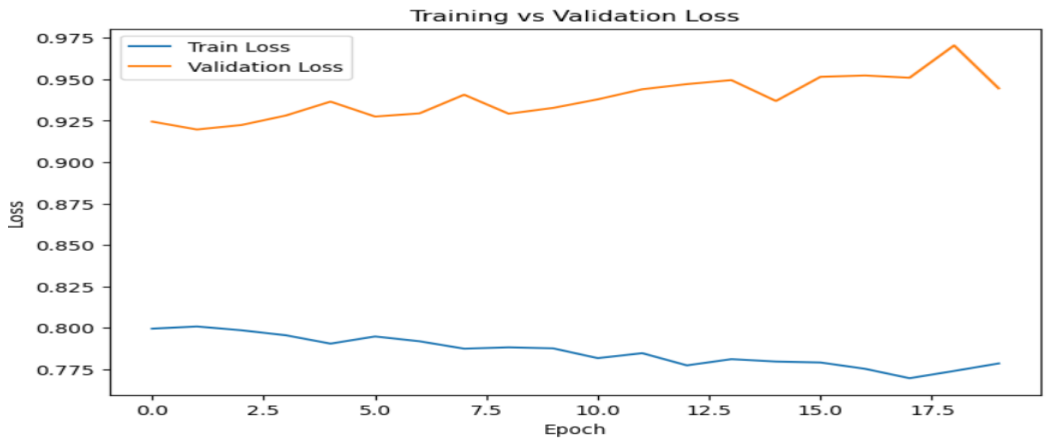
Feed Forward Neural Network

A feed-forward neural network (FFNN) was implemented in PyTorch for binary credit card default prediction. The data was split into training and testing sets using an 80/20 stratified split to preserve the class distribution. All features were standardized using StandardScaler, fit only on the training data to avoid data leakage.

The neural network contained two hidden layers with 64 and 32 neurons, respectively. ReLU activation functions were used after each hidden layer, and batch normalization was applied after the first layer to improve training stability. The output layer consisted of a single neuron for binary classification.

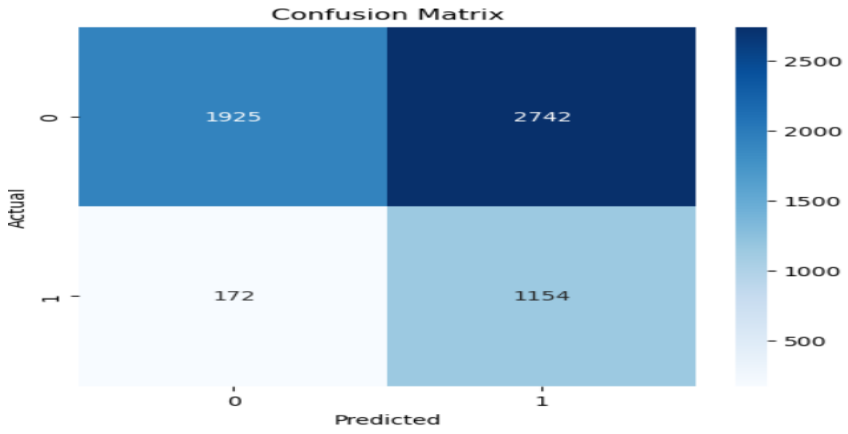
Mini-batch training was performed using a batch size of 32 and the Adam optimizer with a learning rate of 0.001. Since the dataset was imbalanced, BCEWithLogitsLoss was used with a positive class weight equal to the ratio of non-default to default observations. This increased the penalty for misclassifying default cases and helped the model focus more on the minority class.

The model was trained for 20 epochs while tracking both training and validation loss.



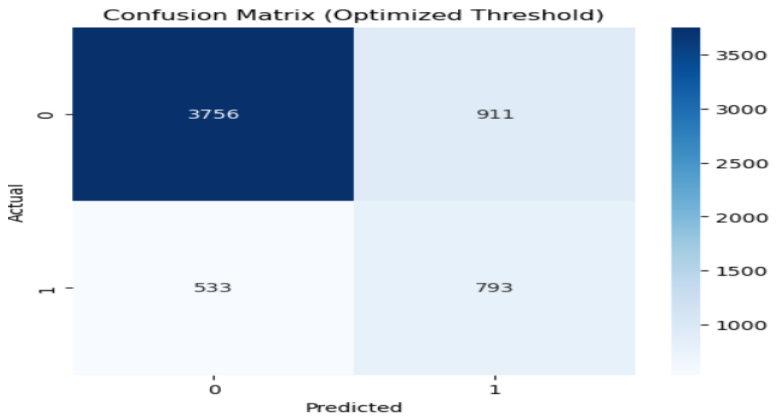
Model Results

The feed-forward neural network achieved a recall of 0.87 for the default class, meaning the model successfully identified most customers who were likely to default. Since the dataset was imbalanced, a lower classification threshold of 0.30 was used instead of the standard 0.50 in order to capture more potential defaulters. This improved recall but reduced precision to 0.30, meaning the model generated more false positives by incorrectly labeling some non-default customers as defaults. In credit risk prediction, prioritizing recall can be beneficial because missing a true default is often more costly than flagging a low-risk customer. The model also achieved an ROC-AUC score of 0.77, indicating a reasonable ability to distinguish between default and non-default customers.



Threshold Adjustment

The neural network was also evaluated using an optimized classification threshold of 0.52, selected to improve the balance between precision and recall through the F1-score. Compared to the lower threshold model, this approach produced a more balanced performance with a precision of 0.47 and a recall of 0.60 for the default class. While fewer defaulters were identified overall, the model reduced the number of false positives and improved the reliability of predicted defaults. The model achieved an F1-score of 0.52 for the default class and an overall accuracy of 0.76, indicating a stronger balance between identifying defaults and minimizing incorrect classifications.



Conclusion

This project explored the use of machine learning models to predict borrower default risk using financial and behavioral data. Four models were evaluated: Naive Bayes, Logistic Regression, a Feed Forward Neural Network, and XGBoost.

The results demonstrate that model performance is strongly influenced by the ability to handle class imbalance and capture meaningful behavioral patterns. Naive Bayes, while effective at identifying most defaulters, produced an excessive number of false positives, limiting its practical use. Logistic Regression provided a stable and interpretable baseline, offering a reasonable balance between precision and recall. The neural network improved recall significantly, but introduced greater model complexity and variability in performance.

Among all models, **XGBoost achieved the best overall results**, combining strong predictive performance with an effective balance between identifying risky borrowers and minimizing incorrect classifications. Its superior performance is largely driven by its ability to model non-linear relationships and leverage engineered features that summarize payment behavior.

A key finding of this project is that **payment history is the most important predictor of default risk**, outweighing demographic characteristics. Additionally, the use of feature engineering and threshold tuning significantly improved model performance, highlighting the importance of both data preparation and post-model optimization.

In credit risk applications, missing a defaulter is more dangerous than falsely flagging one, which is why particular attention was given to the precision–recall tradeoff. However, excessively high false positives can lead to rejecting profitable customers, making balance essential.

Overall, XGBoost provides the most reliable and practical solution for this problem. Its ability to balance risk detection and decision accuracy makes it well-suited for real-world lending scenarios, where both minimizing losses and maintaining profitable customer relationships are critical.

Team Contributions

This project was completed through a collaborative effort, with responsibilities divided among team members based on individual strengths and areas of interest. Each member contributed to multiple stages of the project, including data preprocessing, exploratory data analysis, model development, and report writing.

Matheus Santos Gomes was responsible for data preprocessing, including handling missing values and performing feature engineering. He also implemented and evaluated the Logistic Regression model and contributed to writing both the Exploratory Data Analysis (EDA) and Logistic Regression sections of the report.

Natiana Urdaneta focused on the development and evaluation of the XGBoost model and contributed to writing the conclusion, where she summarized the key findings and insights from the project.

Joaquin Hidalgo-Estrada implemented and evaluated the neural network model, enhancing its performance through techniques such as threshold optimization and class weighting to address dataset imbalance and reduce model bias. He also consolidated all individual code components into a single, organized file prepared for final submission.

Rajan Jinadra contributed to the project by building the Naive Bayes model and helping develop the project's objective and motivation. He also converted the data cleaning code into a clear written section for the final report, making sure the cleaning process was explained properly. In addition, he helped with the final formatting edits of the report to make it more organized, polished, and ready for submission.